2025/2026: 5. Streaming Data Prep in EPL - Bike Sharing and Dutch Energy Generation

Andrea Lancini (10707309)

September 4, 2026

1 Problem and Objective

This project focuses on using the Java version of Esper to process a data stream from a CSV file via EPL (Event Processing Language) queries. The primary dataset is the well-known Bike Sharing Dataset from Washington (UCI). The goal is to implement a full streaming feature engineering pipeline, including temporal transformations, lag features, rolling averages, and exponential moving averages. The pipeline's generalizability is subsequently validated on a second, independent dataset, specifically the Dutch Energy Generation dataset.

2 Architecture

A custom *EventProcessor* class was implemented to instantiate the EPL runtime based on a descriptive mapping between field names and data types. Furthermore, a *compileDeploy* method was established to specify the queries responsible for processing the event stream. This design allows the pipeline to be built modularly by invoking the method multiple times. Once defined, the stream is initiated via the *startStream* function, which generates the events and applies the declared processing pipeline. The main class defines two distinct pipelines tailored to the requirements of the respective datasets.

3 Queries pipeline

The pipeline processes the event stream sequentially, with each query appending new information. The primary steps are outlined below: **Step 1:** Remove instant column.

```
INSERT INTO WithoutInstant
SELECT dteday , hr, weathersit, temp, hum, windspeed, casual, registered
FROM BikeSharingEvent;
```

Step 2: Compute the count field as the sum of casual and registered users.

```
INSERT INTO WithCount
SELECT *, casual+registered as count
FROM WithoutInstant;
```

Step 3: Compute apparent temperature using the Australian apparent temperature formula. Because the original dataset contains data normalized between 0 and 1, an intermediate table is utilized to denormalize the metrics prior to applying the formula.

```
INSERT INTO TrueMetrics
SELECT *, (-8.0 + (47.0)*temp) AS true_temp,
        Math.round(hum*10000)/100 AS true_hum,
```

```
        windspeed*67 AS true_windspeed
FROM WithCount;
INSERT INTO WithATemp
SELECT dteday , hr, weathersit, casual, registered, count,
true_temp as temp, true_hum as hum, true_windspeed as windspeed,
true_temp + 0.33*(true_hum*0.06105*Math.exp((17.27*true_temp)/(237.7+true_temp)))
- (0.7*true_windspeed)-4 AS atemp
FROM TrueMetrics;
```

Step 4: Extract all temporal information from the date field.

```
INSERT INTO WithTemporal
SELECT *, getYear(dteday) as yr, getMonth(dteday) as mnth,
getDayOfWeek(dteday) as weekday, getSeason(dteday) as season,
isHoliday(dteday) as holiday, isWorkingDay(dteday) as workingday
FROM WithATemp;
```

Custom functions (*getYear, getMonth, getDayOfWeek, getSeason, isHoliday, and isWorkingDay*) were implemented to handle specific data encodings, as standard LocalDate return values differ from the integer IDs used in this dataset. The *isHoliday* function relies on the *jollyday* package to verify dates against specific national calendars (the American calendar for the Bike Sharing dataset and the Dutch calendar for the Energy dataset).

Step 5: Compute lagged versions of the count field at lags -1, -24 and -168. This process is repeated for the "registered" and "casual" fields.

```
INSERT INTO WithLag
SELECT *,
count - prev(count, 1) as count_lagged_1,
count - prev(count, 24) as count_lagged_24,
count - prev(count, 168) as count_lagged_168,
FROM WithTemporal.win:length(168);
```

Step 6: Compute the rolling mean over the preceding 3, 6, 12, and 24 hours utilizing the *avg()* operator over a sliding window. The computation is also applied to the registered and casual fields and since the averages are computed for entire windoes, separate queries are defined for different window sizes.

```
INSERT INTO WithRM3
SELECT *, avg(count) AS count_rm_3 FROM WithLag.win:length(3);
INSERT INTO WithRM6
SELECT *, avg(count) AS count_rm_6 FROM WithRM3.win:length(6);
INSERT INTO WithRM12
SELECT *, avg(count) AS count_rm_12 FROM WithRM6.win:length(12);
INSERT INTO WithRM24
SELECT *, avg(count) AS count_rm_24 FROM WithRM12.win:length(24);
```

Step 7: Compute the exponential moving average (EMA) at lag -168. First, an α variable is declared and computed using the following formula:

$$\alpha = \frac{2}{\text{WINDOW\_SIZE} + 1}$$

Next, a state variable is created for each target field to store the previous EMA value, allowing the new value to be computed iteratively:

$$EMA_t = count_t * \alpha + EMA_{t-1} * (1 - \alpha)$$

The corresponding queries are executed as follows:

```
CREATE VARIABLE double globalEmaCount = 0.0;
INSERT INTO FinalStream
SELECT *, (count*alpha) + (globalEmaCount*(1-alpha)) AS count_ema_168,
FROM WithRM24;
ON WithATemp SET globalEmaCount = (count*alpha) + (globalEmaCount*(1-alpha));
```

Step 8: A final selection query is appended to extract the required task-specific data.

4 Results Validation

To ensure the pipeline's accuracy, the values computed in steps 2 and 4 were validated against ground-truth values using a randomly extracted subset of dataset events. Direct dataset validation was not viable for step 3 due to inherent inaccuracies discovered in the source data's apparent temperature values. Consequently, the query results for step 3 were cross-verified via manual calculations, external online calculators and LLM verification.
Step 2: As illustrated in the evaluation of Bike Sharing dataset events, the computed *count* field perfectly matches the pre-existing *cnt* field.

```
{cnt=32, count=32, dteday=2011-01-01, registered=27, casual=5}
{cnt=162, count=162, dteday=2011-02-18, registered=146, casual=16}
{cnt=42, count=42, dteday=2011-04-24, registered=37, casual=5}
{cnt=304, count=304, dteday=2011-05-09, registered=255, casual=49}
{cnt=444, count=444, dteday=2011-05-21, registered=253, casual=191}
{cnt=1, count=1, dteday=2011-07-03, registered=1, casual=0}
{cnt=36, count=36, dteday=2011-07-04, registered=19, casual=17}
{cnt=207, count=207, dteday=2011-08-22, registered=153, casual=54}
{cnt=86, count=86, dteday=2011-12-25, registered=51, casual=35}
{cnt=5, count=5, dteday=2012-01-01, registered=5, casual=0}
{cnt=32, count=32, dteday=2012-01-08, registered=26, casual=6}
{cnt=96, count=96, dteday=2012-01-16, registered=89, casual=7}
{cnt=10, count=10, dteday=2012-01-28, registered=10, casual=0}
{cnt=2, count=2, dteday=2012-02-21, registered=2, casual=0}
{cnt=264, count=264, dteday=2012-03-07, registered=252, casual=12}
{cnt=144, count=144, dteday=2012-04-19, registered=124, casual=20}
{cnt=82, count=82, dteday=2012-05-28, registered=60, casual=22}
{cnt=139, count=139, dteday=2012-06-04, registered=135, casual=4}
```

Step 4: The temporal values extracted by the queries align accurately with the original dataset classifications.

5 Pipeline validation

To demonstrate structural adaptability, the pipeline architecture was replicated for the Dutch Energy Generation dataset. This secondary pipeline requires fewer steps, as the dataset lacks an instant column and the meteorological variables (temperature, humidity, windspeed) necessary for apparent temperature calculation.
The adjusted pipeline execution steps are as follows:

1. Extract temporal features from the date field;

Figure 1: Image showing the results for the fields yr, mnth and weekday

Figure 2: Image showing the results for the fields season, holiday and workingday

2. Compute the total energy produced via fossil sources (*total_fossil*);
3. Compute the lagged iterations of *total_fossil* at lags -1, -96 (1 day), and -672 (1 week);
4. Compute the rolling mean of *total_fossil* on the prior 12 (3 hours), 24 (6 hours), 48 (12 hours) and 96 (1 day) samples;
5. Compute the exponential moving average of *total_fossil* at lag -672 (1 week);

These operations utilize the exact querying logic established in the primary pipeline. The sole architectural difference lies in the implementation of the temporal extraction functions, which were modified to accommodate the distinct data encoding of the Dutch Energy dataset.

6 Code reproducibility

To ensure the reproducibility of this project, the complete source code must be cloned to a local environment. The execution of the pipelines is controlled via a configuration variable named dataset. When running the code, the user must set this variable to select the desired data stream: configuring dataset to 0 initializes the processing pipeline for the Washington Bike Sharing Dataset, whereas setting it to 1 executes the pipeline for the Dutch Energy Generation dataset.